SEMICON 2026 / CHALLENGE BRIEF

PQC Hackathon Challenge

Make an ECC-Based Embedded SPDM Device Quantum-Resistant

Why this challenge matters

Public-key cryptography is entering a formal migration phase. Future quantum computers could break the security assumptions behind widely deployed elliptic-curve cryptography. Data captured today may also be stored and decrypted later, so migration cannot wait until a cryptographically relevant quantum computer exists.

Government security programs, procurement rules, and technical standards are therefore defining transition requirements. CNSA 2.0 is the primary driver for this challenge: for affected U.S. national-security systems and their suppliers, it directs migration away from quantum-vulnerable public-key algorithms such as ECDSA and ECDH toward approved post-quantum algorithms. In practice, an existing ECC-based product cannot simply be replaced overnight. The engineering challenge is to migrate authentication and key establishment while preserving device identity, interoperability, updateability, performance, and constrained-memory operation.

This challenge centers on **SPDM (Security Protocol and Data Model)**, the DMTF standard used to authenticate hardware components and establish secure sessions between them. SPDM is not an embedded niche protocol: it is the attestation and secure-session backbone of the modern **datacenter**. Devices such as GPUs, storage controllers, PCIe/CXL add-in cards, and SmartNICs/DPUs act as SPDM responders that prove their identity and expose signed firmware measurements, while platform and management controllers (BMCs) act as SPDM requesters that verify those measurements and establish authenticated, encrypted channels before a component is trusted on the platform.

SPDM itself is migrating to post-quantum cryptography. **SPDM 1.4 adds PQC support**, defining the use of post-quantum signatures and key establishment (ML-DSA and ML-KEM) alongside the classical ECDSA/ECDHE mechanisms. Because datacenter hardware has a long service life and its attestation identities and firmware trust anchors must remain valid for many years, an SPDM device that authenticates only with ECC today is a future liability: attestation evidence and session key material captured now could be forged or decrypted once a cryptographically relevant quantum computer exists. Making an SPDM endpoint quantum-resistant is therefore directly relevant to securing next-generation datacenter and edge infrastructure — not only to defense use cases. This challenge asks teams to carry an ECC-based SPDM device through exactly this migration on real MCU hardware.

Regulatory and standards context

The challenge is based on a real migration obligation, but participants must distinguish legal requirements from guidance and standards:

- **CNSA 2.0** is the most direct driver for defense and national-security use cases. For this challenge, its applicable algorithm requirements define the target: teams must map the existing ECDSA/ECDH-based flow to the corresponding CNSA 2.0 post-quantum signature and key-establishment requirements and justify any transition-mode exceptions.
- **DMTF DSP2072 “SPDM Post-Quantum Cryptography White Paper”.** Informative, and the most on-point reference for this challenge's PQC migration.
- **U.S. Executive Order 14028 and National Security Memorandum 10** drive federal cryptographic inventory and post-quantum migration planning. They do not require every commercial product to replace ECC immediately, but they affect federal suppliers and procurements.
- **NIST FIPS 203, FIPS 204, and FIPS 205** standardize ML-KEM, ML-DSA, and SLH-DSA. They become particularly relevant when a product or contract requires FIPS-approved algorithms.
- **NIST IR 8547** describes the transition from quantum-vulnerable standards to post-quantum standards and is a planning reference rather than a standalone law.
- **The EU Cyber Resilience Act, NIS2, and DORA** currently create cybersecurity, lifecycle, and risk-management obligations; they do not universally prescribe replacement of ECC with PQC. They strengthen the case for identifying long-lived quantum-vulnerable assets and maintaining a migration-capable product.
- **BSI and UK NCSC guidance** similarly recommends crypto inventory, prioritization, and migration planning for systems with long confidentiality or service lifetimes.

For this hackathon, teams shall state the CNSA 2.0 requirement and algorithm profile they target, show the mapping from the original ECC flow, and identify any remaining classical dependencies

The starting point

The challenge package starts from the following concrete resources:

1. Device and evaluation kit

- **MCU:** PSOC™ Control C3M/P6, using the `PSC3M6GES3A HQ1` device configuration.
- **Evaluation hardware:** PSOC™ Control C3M6 Evaluation Kit, `KIT_PSC3M6_EVAL`.
- **Development environment:** ModusToolbox™ 3.8 or later, with the kit BSP and the supplied host-side tools.
- **Tooling constraint:** teams shall use **Infineon tools and the supplied Infineon software** — ModusToolbox™, its bundled compilers, the Edge Protect tooling, the kit BSP, and the provided SPDM and security code examples.

2. Infineon secure-boot and PQC documentation

Teams shall use these documents as the platform reference:

- **AN243370, Getting started with PSOC™ Control C3x6 MCU security:** secure boot, boot flow, Edge Protect Bootloader, image signing, lifecycle security, firmware upgrade, and key revocation.
- **AN243376, PSOC™ Control C3x6 MCU Provisioning Specification:** OEM root-of-trust provisioning, policy configuration, image signing, re-provisioning, lifecycle transition, and key revocation.
- **AN243402, Getting started with Trusted Firmware-M on PSOC™ Control C3:** TF-M integration, EPBL integration, image signing/programming, secure boot, and image upgrade.
- **PSOC™ Control C3M/P6 MCU Architecture Reference Manual, section 15:** ROM_BOOT, FLASH_BOOT, step-wise image authentication, MCUboot-format images, OEM root-of-trust keys, lifecycle states, and secure-boot behavior.
- **PSOC™ Control C3M/P6 getting-started documentation:** evaluation-kit setup, BSP usage, ModusToolbox workflow, and supported platform cryptography.

The platform documentation identifies **Edge Protect Bootloader (EPBL)** as the boot-stage integration point for PQC image verification. It documents PQC image support and ML-DSA image-validation configuration. Teams must use or extend the provided EPBL so that it verifies the next application image with a PQC or documented hybrid signature before execution. They must state which verification is performed by immutable BootROM and which is performed by EPBL; the immutable BootROM must not be assumed to provide ML-DSA verification unless the selected platform configuration explicitly demonstrates it.

3. Cryptographic and compliance reference

- **CNSA 2.0:** the migration driver and target profile for quantum-resistant signatures and key establishment. Teams must identify the applicable CNSA 2.0 requirements and map them to the selected boot and SPDM algorithms. CNSA 2.0 is a design constraint for this hackathon, not a claim of certification.
- **NIST FIPS 203, FIPS 204, and FIPS 205:** reference standards for ML-KEM, ML-DSA, and SLH-DSA where applicable to the selected implementation.

4. SPDM Code Example

The **SPDM Code Example (SPDM CE)** is the starting real-time communication implementation. It provides:

- an SPDM responder running on the PSoC device;
- a host acting as the SPDM requester;
- ECDSA device authentication and ECC-based certificates/trust anchors;
- ECDHE session key establishment; and
- authenticated and encrypted application data after session establishment.

The existing SPDM CE is functional but uses quantum-vulnerable classical public-key cryptography. It follows the classical SPDM authentication and key-establishment flow. The migration target is the post-quantum capability defined in **SPDM 1.4**, which introduces ML-DSA signatures and ML-KEM key establishment into the protocol. Teams must preserve this use case while migrating both the boot-time image-verification path and the real-time SPDM path to a documented PQC or hybrid architecture aligned with the SPDM 1.4 PQC mechanisms.

Challenge objective

Design, implement, and demonstrate a two-part migration of an embedded device to post-quantum protection:

Boot-time protection: protect the chain of trust from reset through verified application startup, including verification of the next application image.
Real-time protection: protect the device-to-host communication channel using SPDM after the application is running.
Both parts are mandatory. A PQC-protected SPDM session is incomplete if an unauthorized application can boot, and secure boot is incomplete if the running application exposes only a classical or unauthenticated runtime channel.

The CNSA 2.0 algorithm requirements are the design constraint for this challenge. Teams shall identify the relevant CNSA 2.0 requirements for digital signatures, authentication, and key establishment, select the corresponding approved post-quantum algorithms, and implement them in the SPDM responder/requester flow.

The result must protect the same device identity and application communication use case while showing the engineering cost of the migration. A hybrid ECC-plus-PQC flow shall not be part of this challenge.

The final demonstration must show that:

immutable BootROM and the configured boot stages authenticate the intended next image;
EPBL verifies the next application image with the selected PQC signature before execution;
corrupted, incorrectly signed, and unauthorized-rollback images are rejected;
the host can authenticate the PSoC responder after successful boot;
the responder and requester can establish a protected SPDM session;
application data is authenticated and encrypted during real-time operation;
invalid, altered, or non-compliant boot and communication cryptographic material is rejected; and
the security improvement and resource cost of both parts can be measured and explained.

Required implementation tasks

Task 1: Establish the ECC boot and SPDM baseline

Use the supplied ECC-based boot and SPDM implementation as the reference configuration.

The baseline must document the C3P6 boot chain, including ROM_BOOT, FLASH_BOOT, the EPBL or application stage, the OEM root-of-trust key, image format, and lifecycle state.

The baseline must demonstrate boot-time protection:

- authentication of each applicable next-stage image;
- rejection of an altered or incorrectly signed image; and
- the configured development or production lifecycle behavior.

The baseline must also demonstrate real-time protection:

- the PSoC device acting as the SPDM responder;
- the host acting as the SPDM requester;
- responder authentication using ECDSA credentials and certificates;
- session establishment using ECDHE and the existing SPDM flow;
- authenticated and encrypted application data exchange; and
- operation in a development lifecycle state suitable for an open hackathon.

Record the baseline flash usage, RAM usage, session-establishment time, message sizes, and application throughput or latency.

Task 2: Protect boot-time operations

Use or extend the provided EPBL so that the next application image is authenticated with a PQC or documented hybrid signature before execution.

The boot-time implementation must address:

- the separation between immutable BootROM verification and EPBL verification;
- the boot chain and exact point at which each image is authenticated;
- MCUboot image-format metadata, image version, and rollback protection;
- the PQC image-validation key and how it is provisioned or embedded;

- the selected PQC algorithm and its mapping to the stated CNSA 2.0 target;
- rejection of corrupted, incorrectly signed, malformed, and unauthorized-rollback images; and
- the transition from the existing ECC root of trust to a PQC or hybrid boot design.

The C3P6 documentation describes ECDSA-based BootROM image verification and separately identifies ML-DSA/ML-KEM platform support. Teams must not claim that the immutable BootROM is automatically ML-DSA-capable. EPBL that performs PQC verification of the next image, sha also be sufficient PQC protected.

Task 3: Protect real-time operations with SPDM

Extend the baseline so the responder can be authenticated and the session can be established with post-quantum protection.

The implementation should use:

- **ML-DSA** for post-quantum authentication and signatures, as defined by the SPDM 1.4 PQC profile;
- **ML-KEM** for post-quantum key establishment where supported by the selected SPDM/library profile; and
- a certificate, credential, or trust-anchor format that the host can validate.

For the CNSA 2.0 target, use the applicable CNSA 2.0 algorithm profile supported by the selected toolchain and explain the mapping. Do not assume that ML-DSA-65 and ML-KEM-1024 are automatically the correct profile: the team must verify the required security category and algorithm selection for the target CNSA 2.0 system.

The migrated design must address:

- how the host validates the PQC;
- how the session secret is established and derived;
- how credentials and trust anchors are provisioned and updated;
- how large PQC keys, signatures, certificates, and transcripts fit in the MCU memory model;

Task 4: Demonstrate the lifecycle connection

Demonstrate that boot-time and real-time protection form one maintainable device lifecycle.

Implement or describe a secure update path for the application image and SPDM credentials. The update design must show:

- acceptance of a valid signed image or credential update;
- rejection of a corrupted image;
- rejection of an invalid signature;
- protection against rollback to an older unauthorized version; and
- the relationship between the verified boot identity and the SPDM responder identity;
- a migration strategy for devices that initially contain ECC credentials.

The submission must explain whether the runtime SPDM identity is anchored in the verified boot chain, in separately provisioned credentials, or in both, and what security consequences follow. It must clearly identify which root of trust remains classical, which parts are PQC-protected, and what must change before the product can claim full quantum-resistant protection.

Task 5: Present and demonstrate the complete migration

Prepare and deliver a concise technical presentation and live demonstration summarizing the work completed in Tasks 1–4.

The presentation shall cover:

• the original ECC-based architecture and trust model;

• the selected post-quantum algorithms and their mapping to the CNSA 2.0 target profile;

• the secure-boot architecture and the role of BootROM, FLASH_BOOT, and EPBL;

• the migration of SPDM authentication and key establishment to post-quantum mechanisms;

• provisioning, credential management, update, revocation, and rollback protection;

• remaining classical dependencies and limitations;

• resource, performance, and communication impacts compared to the ECC baseline; and

• lessons learned, engineering trade-offs, and recommendations for future migration phases.

The live demonstration shall show:

• successful boot of an authorized image;
• rejection of a modified, invalidly signed, or rollback image;
• successful SPDM responder authentication and protected session establishment;
• authenticated and encrypted application communication;
• at least one tampering attempt and the resulting rejection;
• a valid firmware or credential update; and
• successful reconnect and operation after the update.

Validation scenarios

The demonstration must include positive and negative tests for both the baseline and migrated designs.

Positive tests

- ECC baseline image verification and boot
- PQC or hybrid next-image verification and boot through EPBL
- valid firmware or credential update followed by successful reboot
- ECC baseline SPDM session establishment
- PQC or hybrid SPDM session establishment
- successful responder authentication after boot
- reconnect after update

Negative tests

- modified responder certificate or credential
- invalid signature
- modified MCUboot image metadata or manifest
- unauthorized rollback image
- modified SPDM transcript or handshake message
- unsupported or malformed PQC key material

Measurements and comparison

Report results for the ECC baseline and the PQC.

Category	Required measurement
Security	Boot-stage authentication, image-signature algorithm, SPDM authentication and key-establishment algorithms
Memory	BootROM/EPBL/application flash, boot-time RAM, runtime RAM, credential storage, working buffers, peak handshake memory
Performance	Boot verification time, boot-to-application time, session setup time, signature verification time, encapsulation/decapsulation time
Communication	Handshake message count, total bytes, largest message, fragmentation behavior
Migration	What remains ECC in BootROM/EPBL and SPDM, what becomes PQC, and how an existing deployed device is upgraded

Do not report only the size of an individual key. Explain the total memory impact of certificates, signatures, transcript buffers, temporary computation buffers, metadata, and persistent trust anchors.

Expected deliverables

1. Source code for the EPBL integration, PSoC responder, and host requester/test application.
2. Build and run instructions for ModusToolbox.
3. ECC baseline and PQC implementation configurations.
4. Credential-generation and provisioning scripts.
5. A test report covering boot-time and real-time positive and negative validation scenarios.
6. A resource and performance comparison using the measurements above, separated into boot-time and runtime costs.
7. A short migration architecture describing the path from ECC-only to hybrid and eventually PQC-only operation across both planes.
8. A five-minute live demonstration showing verified boot, protected application traffic, a boot or communication tampering attempt, and the resulting rejection.

Scoring criteria

Every team is scored on the same five criteria. Each criterion is scored from 0 to 5. The challenge-specific criterion counts double (2x), so the maximum is 30 points.

Criterion	Weight	What judges look at
Technical Feasibility	1x	Does the solution work, and is the approach technically sound? Judges look for a working prototype or a credible technical path, and for evidence that is measured, reproducible and honest about its limits.
Innovation	1x	How new is the idea or approach compared with the current state of the art, and how creative is the way it tackles the problem?
Market Potential	1x	Is there a real need, a clear customer or user, and room to grow? Judges look at how large and how reachable the opportunity is.
Business Viability	1x	Could this become a sustainable product or business? Judges look at the business model, the cost and effort of adoption, the route to market and what it would take to get there.
Challenge-specific: Security correctness, migration quality and embedded engineering	**2x**	Boot-time (EPBL) and real-time (SPDM) post-quantum protection shown through positive and negative tests, mapped to CNSA 2.0, with remaining classical dependencies and the measured cost against the ECC baseline made explicit, on hardware that fits the PSoC resource constraints.

Total: 6 weighted units (1 + 1 + 1 + 1 + 2), maximum 30 points.

Challenge-specific criterion: Security correctness, migration quality and embedded engineering (2x)

Security correctness

- The device boots only an authorized image and rejects corruption, invalid signatures, and unauthorized rollback.
- The responder is authenticated by the requester.
- The session provides authenticated encryption for application data.
- Invalid boot credentials, runtime credentials, signatures, transcripts, and downgrade attempts are rejected.
- The implementation does not silently fall back to an insecure classical-only mode.

Migration quality

- The design addresses a real ECC-to-PQC transition problem.
- CNSA 2.0 or another stated compliance driver is mapped to concrete technical changes.
- The design identifies transition limitations and remaining classical dependencies.
- Provisioning, rotation, revocation, firmware update, boot authentication, lifecycle transition, and rollback are considered.

Embedded engineering

- The implementation fits the PSoC resource constraints or clearly documents tradeoffs.
- Large PQC objects are handled safely without buffer overflows or uncontrolled memory use.
- The code is reproducible, understandable, and testable.